

# Institut Supérieur d'Informatique et de Multimédia de Gabès

Année Universitaire : 2025/2026

Module : Système d'exploitation 2

Enseignante : Y. BEN JMAA

## TP2 : Gestion des processus

Travaux Pratiques avec corrections détaillées

# Table des matières

|          |                                                                 |          |
|----------|-----------------------------------------------------------------|----------|
| <b>1</b> | <b>Objectifs</b>                                                | <b>2</b> |
| <b>2</b> | <b>Rappel des primitives système</b>                            | <b>2</b> |
| <b>3</b> | <b>Exercice 1</b>                                               | <b>2</b> |
| 3.1      | Programme fork1.c . . . . .                                     | 2        |
| 3.2      | Compilation . . . . .                                           | 2        |
| 3.3      | Exécution . . . . .                                             | 2        |
| 3.4      | Explication du résultat . . . . .                               | 2        |
| <b>4</b> | <b>Exercice 2</b>                                               | <b>3</b> |
| 4.1      | Programme fork2.c (corrigé) . . . . .                           | 3        |
| 4.2      | Compilation . . . . .                                           | 3        |
| 4.3      | Exécution . . . . .                                             | 3        |
| 4.4      | Explication du résultat . . . . .                               | 3        |
| <b>5</b> | <b>Exercice 3</b>                                               | <b>4</b> |
| 5.1      | Programme fork3.c . . . . .                                     | 4        |
| 5.2      | Compilation . . . . .                                           | 4        |
| 5.3      | Exécution . . . . .                                             | 4        |
| 5.4      | Nombre de processus lancés . . . . .                            | 4        |
| <b>6</b> | <b>Exercice 4</b>                                               | <b>4</b> |
| 6.1      | Programme fork4.c (non synchronisé) . . . . .                   | 4        |
| 6.2      | Compilation . . . . .                                           | 5        |
| 6.3      | Exécution et observation . . . . .                              | 5        |
| 6.4      | Version synchronisée (affichage 1 à 100 dans l'ordre) . . . . . | 5        |
| <b>7</b> | <b>Exercice 5</b>                                               | <b>6</b> |
| 7.1      | Programme corrigé . . . . .                                     | 6        |
| 7.2      | Questions et réponses . . . . .                                 | 7        |
| <b>8</b> | <b>Conclusion</b>                                               | <b>8</b> |

## Objectifs

- Utilisation de la primitive de base : `fork()` pour la création d'un processus
- Compilation et exécution des programmes en langage C

## Rappel des primitives système

- `fork()` : duplique le processus
  - Retourne 0 pour le processus fils
  - Retourne PID (fils) pour le processus père
  - Retourne -1 en cas d'échec
- `getpid()` : retourne l'identificateur du processus courant
- `getppid()` : retourne l'identificateur du processus père
- `sleep(int sec)` : le processus appelant s'endort pour `sec` secondes
- `wait(int *status)` : suspend l'exécution du processus appelant jusqu'à ce que ses processus fils se terminent
- `exit(int status)` : termine un processus

## Exercice 1

### Programme fork1.c

```
1 #include <stdio.h>
2 #include <unistd.h>
3 int main()
4 {
5     pid_t pid;
6     pid = fork();
7     printf("hello je programme\n");
8     return(0);
9 }
```

Listing 1 – Programme fork1.c

### Compilation

```
gcc fork1.c -o f1
```

### Exécution

```
./f1
```

### Explication du résultat

- Le message "hello je programme" s'affiche **deux fois** :
- Une fois par le processus père
  - Une fois par le processus fils

**Explication :** Après l'appel à `fork()`, le processus fils commence son exécution à partir de l'instruction suivante (`printf`). Les deux processus exécutent donc la même instruction.

## Exercice 2

### Programme fork2.c (corrigé)

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <stdlib.h>
4 #define result 7
5 int main()
6 {
7     pid_t pid;
8     switch(pid = fork())
9     {
10         case -1:
11             perror("cr ation impossible");
12             exit(1);
13         case 0:
14             printf("%d_fils_de_%d\n", getpid(), getppid());
15             exit(result);
16         default:
17             printf("%d_p_re_de_%d\n", getpid(), pid);
18             wait(NULL);
19     }
20     return 0;
21 }
```

Listing 2 – Programme fork2.c corrigé

### Compilation

```
gcc fork2.c -o f2
```

### Exécution

```
./f2
```

### Explication du résultat

Sortie possible :

```
1234 père de 1235
1235 fils de 1234
```

- Le père affiche son PID et celui du fils
- Le fils affiche son PID et celui du père
- Le fils termine avec `exit(7)`
- Le père attend la fin du fils avec `wait(NULL)`

**Remarque :** Pour récupérer la valeur de retour du fils, il faudrait utiliser `wait(&status)`.

## Exercice 3

### Programme fork3.c

```
1 #include<stdio.h>
2 #include<unistd.h>
3 #include<sys/types.h>
4 int main()
5 {
6     pid_t pid1, pid2;
7     pid1 = fork();
8     pid2 = fork();
9     pause();
10    return(0);
11 }
```

Listing 3 – Programme fork3.c

### Compilation

```
gcc fork3.c -o f3
```

### Exécution

```
./f3
```

### Nombre de processus lancés

**Réponse : 4 processus**

**Explication :**

- Premier `fork()` : le processus original P crée F1
- Second `fork()` : P crée F2, et F1 crée F3
- Chaque processus appelle `pause()` et reste bloqué

**Arbre des processus :**

```
      P
     / \
    F1  F2
   /
  F3
```

Total = 4 processus

## Exercice 4

### Programme fork4.c (non synchronisé)

```
1 #include<stdio.h>
2 #include<unistd.h>
3 #include<sys/wait.h>
4 #include<stdlib.h>
5
6 int main()
7 {
8     pid_t pid1, pid2;
9     pid1 = fork();
10
11     if (pid1 == 0) {
12         for (int i = 1; i <= 50; i++)
13             printf("%d_", i);
14         exit(0);
15     } else {
16         pid2 = fork();
17         if (pid2 == 0) {
18             for (int i = 51; i <= 100; i++)
19                 printf("%d_", i);
20             exit(0);
21         } else {
22             wait(NULL);
23             wait(NULL);
24         }
25     }
26     return 0;
27 }
```

Listing 4 – Programme fork4.c non synchronisé

## Compilation

```
gcc fork4.c -o f4
```

## Exécution et observation

```
./f4
```

**Observation :** Les nombres de 1 à 100 sont affichés dans un **ordre non séquentiel** car les deux fils s'exécutent en parallèle.

## Version synchronisée (affichage 1 à 100 dans l'ordre)

```
1 #include<stdio.h>
2 #include<unistd.h>
3 #include<sys/wait.h>
4 #include<stdlib.h>
5
6 int main()
7 {
```

```

8   pid_t pid1, pid2;
9   pid1 = fork();
10
11  if (pid1 == 0) {
12      for (int i = 1; i <= 50; i++)
13          printf("%d_", i);
14      exit(0);
15  } else {
16      wait(NULL); // Attend fin du premier fils
17      pid2 = fork();
18      if (pid2 == 0) {
19          for (int i = 51; i <= 100; i++)
20              printf("%d_", i);
21          exit(0);
22      } else {
23          wait(NULL); // Attend fin du deuxi me fils
24      }
25  }
26  printf("\n");
27  return 0;
28 }

```

Listing 5 – Programme fork4.c synchronisé

**Explication :** Le père attend la fin du premier fils avant de créer le second, ce qui assure un affichage séquentiel.

## Exercice 5

### Programme corrigé

```

1  #include <stdio.h>
2  #include <unistd.h>
3  #include <sys/wait.h>
4  #include <stdlib.h>
5
6  int main()
7  {
8      pid_t rf;
9      printf("\nLe p re_d marre\n");
10     rf = fork();
11     if (rf > 0)
12     {
13         printf("\nJe suis le p re_\n");
14         printf("\nJe suis en attente de mon fils\n");
15         wait(NULL);
16         printf("\nMon fils a termine_\n");
17     }
18     else if (rf == 0)
19     {

```

```

20     printf("\nJe suis le fils\nMon PID est: %d", (int) getpid()
21           );
22     printf("\nJe vais dormir pour 10 secondes\n");
23     sleep(10);
24 }
25 else
26     printf("\nErreur d'execution de fork\n");
27 return 0;
28 }

```

Listing 6 – Programme exercice 5 corrigé

## Questions et réponses

### 1. Compiler et exécuter. Interpréter le résultat obtenu.

- Le père attend la fin du fils avec `wait()`
- Le fils dort 10 secondes puis se termine
- Le père affiche "Mon fils a termine" après la fin du fils

### 2. Exécuter en arrière-plan. Lancer `ps -f`. Interpréter.

```

./f5 &
ps -f

```

On voit deux processus (père et fils). Le fils est en `sleep`, le père en attente.

### 3. Supprimer `wait()`. Compiler et exécuter. Observation ?

- Le père termine immédiatement
- Le fils devient **zombie** jusqu'à sa fin

### 4. Lancer sans `wait()` en arrière-plan. `ps -f`. Observation ?

```

./f5 &
ps -f

```

Le fils apparaît comme `<defunct>` (zombie) après sa fin.

### 5. Ajouter une variable globale. Modifier et afficher dans père, fils et fin.

```

1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/wait.h>
4 #include <stdlib.h>
5
6 int var = 100;
7
8 int main()
9 {
10     pid_t rf;
11     printf("Variable initiale: %d\n", var);
12     rf = fork();
13     if (rf > 0)
14     {
15         var = 200;

```



```
16     printf("P re_modifie_var : %d\n", var);
17     wait(NULL);
18     printf("P re_fin, var= %d\n", var);
19 }
20 else if (rf == 0)
21 {
22     var = 300;
23     printf("Fils_modifie_var : %d\n", var);
24     sleep(2);
25     exit(0);
26 }
27 return 0;
28 }
```

**Interprétation :** Chaque processus possède sa **copie indépendante** de `var` après `fork()`. Les modifications ne sont pas visibles entre père et fils.

## Conclusion

Ce TP permet de maîtriser :

- La création de processus avec `fork()`
- La distinction père/fils via la valeur de retour
- La synchronisation avec `wait()`
- L'observation des processus avec `ps`
- La notion de mémoire dupliquée après `fork()`
- Les états des processus (actif, zombie, etc.)

